

CS301

DATA STRUCTURE AND ALGORITHMS

LECTURE 15: HASH TABLE

Pandav Patel  
Assistant Professor

Computer Engineering Department  
Dharmsinh Desai University  
Nadiad, Gujarat, India

# OBJECTIVE

- To understand hash table and time complexities of its operations
- To understand importance of choosing a right hash function
- To understand collision and resolution techniques

# OVERVIEW

## 1 OBJECTIVE

## 2 INTRODUCTION

- A problem statement
- Solution
- Drawback of the solution
- How to overcome this drawback?
- Example

## 3 HASH FUNCTION

- Properties
- Mode (a.k.a. division) method
- Multiplication method
- Multiply-Add-Divide (MAD) method
- More on hash functions

## 4 COLLISION RESOLUTION TECHNIQUES

- Chaining
- Open addressing

## A PROBLEM STATEMENT

- I want to store integer numbers such that I can efficiently perform insert, delete and search operations. I also want to make sure that number is not repeated.
  - What data structure would you choose?
    - Array/linked list? Sorted/unordered?

|                      | insert       | delete       | search       |
|----------------------|--------------|--------------|--------------|
| Sorted array         | $O(n)$       | $O(n)$       | $O(\log(n))$ |
| Unsorted array       | $O(n)$       | $O(n)$       | $O(n)$       |
| Sorted linked list   | $O(n)$       | $O(n)$       | $O(n)$       |
| Unsorted linked list | $O(n)$       | $O(n)$       | $O(n)$       |
| Balanced BST         | $O(\log(n))$ | $O(\log(n))$ | $O(\log(n))$ |

- Insert operation in case of unsorted array and unsorted linked list takes  $O(n)$  time if keys need to be unique.  $O(1)$  otherwise.

## SOLUTION TO THE PROBLEM

- Say range of numbers be 0 to  $r - 1$ , maximum numbers to be stored is  $n$  (which is  $\leq r$ )
  - What if  $r$  is small (close to 1000)? Can we create a boolean array ( $arr$ ) of size  $r$  instead of integer array of size  $n$ ?
    - Initialize all the elements of boolean array with value 0
    - To insert  $x$ , change value of  $arr[x]$  to 1
    - To search  $x$ , check value of  $arr[x]$ . Value 0, means not present. Value 1 means present
    - To delete  $x$ , change value of  $arr[x]$  to 0
  - This is called *direct addressing*
  - Now we can perform all three operations (insert, delete and search) in constant time

## DRAWBACK OF THE SOLUTION

- What is the drawback of the discussed solution?
  - It is impractical for larger value of  $r$ . Say  $10^{18}$  (roughly  $2^{60}$ )
  - It will require 131,072 Tera-byte (TB) of memory even if we use only one *bit* for a number in the range
  - It leads to waste of memory

## HOW TO OVERCOME THIS DRAWBACK?

- Instead of allocating memory for all the elements from 0 to  $r - 1$ , can we have only  $m$  slots of memory, and somehow map range (0 to  $r - 1$ ) to (0 to  $m - 1$ )? Here  $m$  is  $O(n)$  - very small compared to  $r$
- A function will be used to map large range into smaller (manageable) range. Such function is called **hash function**
  - Can you think of any function which can be used here?
  - Can more than one number from 0 to  $r - 1$  map to the same slot in 0 to  $m - 1$  range? Yes. *Pigeonhole principle*.
    - It is called **collision**
    - How to handle it?
    - One commonly used technique is **chaining** - Each slot holds address of first node of linked list (could be array or root of BST) which will store numbers that are mapped to that slot by hash function
    - Please note array contains address now and it is not a boolean array

|                                 |                                |
|---------------------------------|--------------------------------|
| OBJECTIVE                       | A PROBLEM STATEMENT            |
| INTRODUCTION                    | SOLUTION                       |
| HASH FUNCTION                   | DRAWBACK OF THE SOLUTION       |
| COLLISION RESOLUTION TECHNIQUES | HOW TO OVERCOME THIS DRAWBACK? |
|                                 | EXAMPLE                        |

## EXAMPLE

- $r: 2^{30}$
- $m: 10$
- $n: 5$
- Elements to store: 192030203, 12, 4353, 45, 345345342
- Hash function used to figure out slot for  $x$  is  $h(x) : x \bmod m$



## PROPERTIES OF HASH FUNCTION

- Hash function ( $h$ ) must be deterministic, means for a value of  $x$ ,  $h(x)$  should always be same
- If a function is not deterministic then it can not be used as hash function
- Properties of good hash function:
  - Efficient to compute (execution in constant time)
  - Uniform distribution of keys to indices of hash table
- Say you want to store 1000 mobile numbers each of 10 digits. Which of the following hash function would be a better choice?
  - $\text{mobile\_num} / 100000000$
  - $\text{mobile\_num} \bmod 1000$
- Ideally, hash function should consider all bits of the key
- Efficiency of hash table critically depends on choice of hash function.

## MODE (A.K.A. DIVISION) METHOD

- $h(k) = k \bmod m$ 
  - Where  $k$  is key, and  $m$  is size of hash table
- What should be the table size?
  - Power of 2?
  - Prime number?
  - Prime number is a better choice. Power of 2 considers only last bits of the number and ignores initial bits.
    - Experimental results show that prime number which is not close to power of 2 results in less collisions compared to prime number which is close to power of 2

## MULTIPLICATION METHOD

- Multiply key with real constant  $c$
- Extract fractional part of  $k * c$
- Multiply result with table size and take its floor
- $h(k) = \text{floor}(m * \text{frac}(k * c))$
- Some values of  $c$  work better compared to other values
  - e.g.  $(\text{sqrt}(5) - 1)/2 = 0.6180339887$  works very well
    - For above value of  $c$  and  $m = 32$ , index for key 12345 will be calculated as
    - $\text{frac}(12345 * 0.6180339887) = 0.6295905015$
    - $32 * 0.6295905015 = 20.1468960480$
    - $\text{floor}(20.1468960480) = 20$
- For this method, choice of  $m$  is not critical. It can be any value, including power of 2.

## MULTIPLY-ADD-DIVIDE (MAD) METHOD

- Given hash function can you generate multiple keys which hash to same slot in hash table?
  - Yes, as far as  $r$  (possible keys) is larger than  $m$  (slots in hash table). There must be at least one slot with  $r/m$  or more possible keys (from 0 to  $r - 1$ ) mapping to it
  - So an adversary (problem setter in case of coding competition) can hack the system if s/he knows hash function in advance.
    - Hash function used in STL is there for anyone to peek into it. This [link](#) explains how it is exploited by problem setters to design test case with inputs which hash to the same slot in hash table

## MULTIPLY-ADD-DIVIDE (MAD) METHOD (CONT...)

- What is the solution to the discussed problem?
  - Can we use set of hash functions and pick one at random during execution. Then use that hash function till completion of the program
  - $h_{ab}(k) = (a * k + b) \bmod m$ 
    - Value of  $a$  and  $b$  is picked at random at the beginning and remains same for a hash table
    - Above hash function is a family of hash functions as different values of  $a$  and  $b$  will generate different hash
    - Now adversary can not come up with keys which hash to same slot in the hash table
    - $a$  must also not be zero. Why?
    - $a$  should not be multiple of  $m$ . Preferably they should be co-prime. Otherwise output of hash function will only depend on  $b$  and hence all keys will map to same slot.
    - For following (one of many) **universal hash function**, where  $p$  is prime number and  $p > r$ , value of  $a$  and  $m$  does not have to be co-prime as far as  $a$  is fairly large number
    - $h_{ab}(k) = ((a * k + b) \bmod p) \bmod m$

## MORE ON HASH FUNCTIONS

- Read about following hash functions on your own
  - Mid-square method
  - Folding method
- What if key is not integer?
  - What if it is float or string?
  - First convert it to int and then map it to index of the hash table
  - In this case hash function has two parts
    - Hash code map - Convert non-int key to int
    - Compression map - Map larger range to smaller range (what we did so far)
- Side note: Note that hash table can also have value associated with the each key

## LOAD FACTOR AND VARIANCE

- What if you keep adding keys and number of entries stored in hash table ( $n$ ) becomes much larger than slots in hash table ( $m$ )?
  - Size of the hash table needs to increase and all  $n$  entries need to be rehashed to be stored in new table with larger size. Same as `std::vector` when memory is relocated to new location
  - When to increase the capacity of hash table?
    - When **load factor** ( $n/m$ ) becomes more than a fixed value.
    - That fixed value changes from implementation to implementation
    - Say it is 0.75 and hash table has room for 16 elements, its capacity will be increased to 32 elements when 13<sup>th</sup> element is stored. Because  $16 * 0.75 = 12$
- Another very useful metric for measuring quality of hash function is variance of number of entries per slot in the hash table

# CHAINING

- Already discussed in previous slides
- How to insert, delete and search for an entry?
  - How efficient will these operations be?
  - For a good hash function that uniformly distributes the keys to slots in hash table, efficiency of these operation will be order of load factor
  - Can load factor be more than 1 in case of chaining? Yes, it can be.



## OPEN ADDRESSING: LINEAR PROBING

- What if we want to store all the elements in the hash table itself instead of creating separate linked list? Such collision resolution techniques are known as **open addressing** techniques
- One such technique is **linear probing**
  - If a slot is full (in case of collision) we can store that element in next available slot in the hash table
  - How will insert and search work?
  - How will delete work?
- Can load factor be more than 1 in case of open addressing? No, it must be less than or equal to 1. Why?

## OPEN ADDRESSING: LINEAR PROBING (CONT...)

- Example:
  - $h(k) = k \bmod 7$
  - insert 2, 15, 3, 16
  - delete 3
  - search 16
  - insert 23
- Initially all slots of hash table are **null**
- **Tombstone** (special value) is used to mark deleted key
  - Why is tombstone needed?
  - What will search and insert do when they encounter **tombstone**?
- Read about *quadratic probing* on your own

## OPEN ADDRESSING: DOUBLE HASHING

- Linear probing sometimes leads to cluster of keys and its performance deteriorates
- **Quadratic probing** can be used to address it
- Another technique that can be used to avoid clustering of keys is **double hashing**
  - It used two hash functions - **one for figuring out index** (slot) from where to start probing and other hash function to **decide the offset by which to increment the index before next probe**. For example
    - $h1(k) = k \bmod m$
    - $h2(k) = 1 + (k \bmod x)$
  - For **linear probing value of  $h2(k)$  is always 1**. So it checks all slots before declaring hash table as full
  - If offset (output of second hash function) divides size of table ( $m$ ) then all the locations will not be visited and it will declare table full even if it is not
  - If  $m$  is prime, this technique will eventually visit all the locations before declaring it full

## OPEN ADDRESSING: DOUBLE HASHING (CONT...)

- $h1(k) = k \bmod 13$
- $h2(k) = 1 + (k \bmod 5)$
- Insert 5, 17, 18, 22, 24, 19, 31

- How will search and delete work?
- Will tombstone we needed?



